

Software Quality Engineering (SE3002)

Assignment 02 — 100 marks

Structural Testing and Coverage Analysis of PX4 Autopilot v1.17.0 — SE3002 Assignment 02

PX4-Autopilot v1.17.0 · d6f12ad1c4f70ad3230afd7d86e971421e02fef4

Ali Haider Bajwa (24i3102) · Taimoor Khalid (24i3052) · Ashar Ahmed
(24i3072)

Section C (SE)

1. What this report covers

We structurally tested four production areas of PX4 v1.17.0, wrote 95 tests of our own against them, measured statement and branch coverage before and after, and performed MC/DC analysis on the failsafe-critical Battery component. No production code was changed at any point. This report records the group, the machine and baseline everything traces to, why these three areas were chosen, how each test was derived from the production logic, what the MC/DC evidence shows, what coverage was achieved, what gaps remain and why, and a final judgment on what the evidence does and does not prove.

2. Environment and fixed baseline

Everything was cloned, built, tested, and measured locally on a student-owned machine. The table below is the complete environment record.

Item	Value
OS	Linux Mint 22.3 (Ubuntu 24.04 base)
Hardware architecture	x86_64
Compiler	gcc / g++ 13.3.0
Build tools	cmake 3.28.3, ninja 1.11.1
Coverage tools	lcov 2.0-1 (gcov 13.3.0)
Other tools	Python 3.12.3, git 2.43.0
Repository	`https://github.com/PX4/PX4-Autopilot.git`, cloned recursively at tag `v1.17.0`
Fixed commit	`d6f12ad1c4f70ad3230afd7d86e971421e02fef4`

Every coverage number in this report traces to that commit plus our own test-only changes. One build note: the top-level `make tests` wrapper was bypassed because its build macro mishandles paths, and we configured and built directly with `cmake` and `ninja` instead (§10). This is only a difference in how the build is invoked, not a change to any code. The upstream baseline runs green before our tests were added: `unit-Hysteresis` passes 7 of 7 and `unit-PID` passes 4 of 4, with the logs kept in `evidence/baseline/`.

3. Scope selection: the three areas and why they qualify

We read the production code first and selected four areas. Each one is real control or business logic with decisions worth testing — not a wrapper, not branch-free math, and not generated or GUI code. The Battery file is a large, substantial component (497 lines) covering state-of-charge fusion, internal-resistance estimation, warning and fault classification, and remaining-time estimation; the Gyroscope calibration file (284 lines) decides which calibration every gyro flies with — device selection, slot binding, parameter load/save with validity guards, and

thermal-offset correction. Across the four areas we exercise 386 lines of production logic, the great majority of it inside Battery's failsafe paths and Gyroscope's calibration chain.

#	Production area	What it does, in plain words	Why it is worth testing	Size	Test level
1	`src/lib/hysteresis/hysteresis.cpp`	A boolean switch with separate on/off time delays, driven by caller-supplied timestamps	A genuine two-sided time-guarded state machine with two compound direction guards; fully deterministic, ideal for unit testing	60 lines	GTest unit
2	`src/lib/pid/PID.cpp`	A control-loop PID with output and integral saturation, anti-windup, and guards against invalid inputs	Sits on a control-critical path with saturation, conditional integration, and degenerate-input guards; deterministic through its direct API	47 lines	GTest unit
3	`src/lib/battery/battery.cpp`	Fuses voltage-based and coulomb-counted charge, estimates internal resistance, classifies warnings across a four-rung ladder, detects faults, estimates remaining flight time	Directly drives failsafe behavior (a missed EMERGENCY warning risks the vehicle); depends on parameters, uORB topics, and time, which is exactly what functional testing exists for; no upstream test links this library, so its baseline coverage was	497 lines	GTest functional

			zero		
4	`src/lib/sensor_calibration/Gyroscope.cpp`	Decides which calibration each gyroscope flies with: device selection, calibration-slot binding, parameter load/save with validity guards, thermal-offset correction from uORB	Calibration correctness feeds attitude estimation, so it is flight-safety logic; depends on parameters and the sensor_correction topic (functional level); no upstream test covers this library, so its baseline coverage was zero	284 lines	GTest functional

Deliberately excluded: SlewRate (branch-free math with nothing to decide), circuit_breaker (a single superficial wrapper of the kind the assignment rejects), and all GUI, generated, and test-only code. Difficult logic was never excluded to flatter a percentage — Battery's estimator and failsafe chains are the hardest logic in the scope and they are the center of it.

4. How the 61 tests were derived from the production code

We started from the source, not from a coverage target. Every `if`, `&&`, and `||` in the four files became a decision that must be shown both true and false. Every boundary comparison (`>=`, `<`, exact constants such as the 1.05 over-voltage factor, the 0.01 offset-epsilon, the calibration-index bounds) became a boundary test at the exact value and one step on either side. Every invalid or error path (NaN, infinity, zero cells, unconnected battery, stale timestamps, unbound calibration slots, out-of-range desired slots, unadvertised topics) became an error or robustness test. The resulting suites and the decisions they discharge are summarized below; the per-test record (95 rows: HST-01-11, BST-01-33, PST-01-17, GST-01-34) is workbook Sheet 1.

Suite	Level	Tests	Decisions covered	Result
HysteresisStructural	Unit	11	H-D1-H-D8, including exact <code>>=</code> -expiry boundaries the upstream suite	11/11 pass

			never probes	
`BatteryStructural`	Functional	33	B-D1–B-D12: warning ladder, spike-fault detection, estimator-reset gate, charge fusion, time-delta clamp, scale fallback, publish paths, armed and fixed-wing remaining-time gates	33/33 pass
`PIDStructural`	Unit	17	P-D1–P-D3 plus saturation edges, update-integral flag, NaN/negative- infinity guards, derivative-guard cases, exact `FLT_EPSILON` boundary, negative-dt robustness, windup clamping, limit edges, zero- gain configuration	17/17 pass
`GyroscopeStructural`	Functional	34	G-D1–G-D13: device selection (internal/external determinism via bus-type bits), correction-slot lookup across all four slots, offset epsilon and finiteness guards, index bounds, parameter load validity guards, save slot search with out-of-range and no-slot cases, rotation round- trip math	34/34 pass

Authorship note for the PID file: it was derived and authored by Ashar Ahmed (branch `ashar-pid-work`, with a living handoff document) and independently verified on Linux by the lead, who corrected five expectations against the v1.17.0

source before the final green run. The original delivery is preserved untouched for provenance.

5. Test execution: results and the eight failures

All six binaries build and run in both the normal and the instrumented coverage configurations. Baseline: upstream unit-Hysteresis 7/7, upstream unit-PID 4/4. Student suites: 11/11, 17/17, 33/33, and 34/34 for the new Gyroscope suite, which passed every test from its first green run through all three coverage rounds. The logs in evidence/logs/ and evidence/baseline/ show every one of these runs. No failure was manufactured and none was hidden: eight development-time failures occurred, each was investigated (setup, expected values, and dependency behavior checked before any defect was suspected), and each turned out to be a wrong expectation on the test side — two inverted models of the estimator-reset gate's pre-initialization firing, one NaN output where IEEE semantics produce NaN for infinite feedback while the integral guard itself held, two D-term sign errors against the PLUS in PID line 47, and three saturation expectations written against a freeze gate that does not exist in v1.17.0. Production code was not touched in any of the eight corrections.

6. MC/DC analysis of the critical component

MC/DC means proving that each individual condition inside a compound decision can independently flip that decision's outcome — that no condition is dead or masked. Battery was chosen for this analysis because its warning and fault classification directly drives failsafe behavior. Five substantive compound decisions implement or directly arm that behavior, and all five were analyzed with independence pairs. Two-condition hysteresis and PID guards are additionally shown as supporting condition/decision evidence, not as the MC/DC claim. Workbook Sheet 2 carries the full per-pair record (32 rows): atomic-condition values, overall outcome, the pair, the demonstrated condition, and the source/test reference.

Decision	Location	Conditions	What the pairs prove
Estimator-reset gate (B-D4)	line 130	4 (connected, initialized, resistance flag, cell count)	Each condition flipped while the other three hold; open-circuit estimate 16.0 vs 0.0 with zero current
Over-voltage fault (B-D8)	lines 327–328	2 (limit set, spike above 1.05× limit)	Fault shown present and absent, including the exact 1.05 boundary proving the

			comparison is strict
Filter-reset disjunction (B-D9, conditions 2-3)	lines 370-371	2 shown of 3	Near-zero-state and consumed-transition pairs; a moved reset target (5.0 → 7.0) discriminates a genuine hold from a re-reset
Armed-update guard (B-D10)	line 375	2 (armed, finite current)	Unarmed-vs-armed and finite-vs-NaN pairs on the remaining-time update path
Fixed-wing level-flight gate (B-D11)	lines 377-378	3 (fixed-wing, fresh stamp, level flight)	Multirotor-vs-fixed-wing, fresh-vs-stale, level-vs-climb pairs
Hysteresis direction guards (supporting)	H-D5, H-D7	2 each	Armed-vs-quiescent and fired-vs-held pairs at exact expiry boundaries
PID derivative guard (supporting)	P-D3	time × history	Time-by-history cases with the exact epsilon boundary

Reaching each condition required explicit setup, all of it visible in the test bodies: synthetic timestamps, direct parameter control, and fixture-held uORB publications so subscriptions actually receive data. (A real discovery along the way: helper-local publications unadvertise on destruction and silently break subscriptions — a test-design defect we fixed by holding publications in the fixture. This is the correct use of test doubles the course teaches: the doubles serve the production logic, not the other way round.) The conclusion of the analysis is that every atomic condition in this failsafe-critical classification logic can independently decide its outcome on the real code path — except the two cases in §7 that are unreachable by construction rather than by weak testing. The Gyroscope file's compound gates are all simple two-condition guards, so no new MC/DC claim is made for it: its 34 tests provide statement, branch, and condition/decision evidence, and the submission's MC/DC claim rests entirely on Battery's five substantive decisions above.

7. Coverage results: baseline versus final

Coverage was captured with lcov and gcov branch coverage, once before our tests (baseline) and once after (final). Both machine-readable reports and the full final HTML are submitted; the table states the key line and branch results per scope item.

Scope item	Baseline	Final lines	Final branches
Hysteresis	22/22 lines, 17/20 arcs	22/22	17/20 (3 arcs provably

	(upstream suite)		unreachable, §8)
PID	22/22 lines, 8/10 arcs (upstream suite)	22/22	10/10 (both gaps closed)
Battery	absent — no upstream test links the library	230/231 (99.6%), all 19 functions	174/248 (see §8 for the accounting)
Gyroscope	absent — no upstream test covers this library	112/112 (100%), all 12 functions	112/145 (see §8 for the accounting)

Our contribution is therefore the entire Battery and Gyroscope coverage from a zero baseline on both, plus the hysteresis boundary and MC/DC obligations and the two PID sub-cases (non-finite integral, derivative guard) that the upstream baseline left open. The calibration helper `Utilities.cpp` is a dependency, not a scope item: it receives incidental coverage (80/112 lines) from the Gyroscope tests, which is reported here for completeness and claimed for nothing.

8. Findings, gaps, limitations, and improvements

Defects found: none reproducible in production code. The eight development failures in §5 were all test-side expectation errors, corrected without touching production code, and no defect is manufactured to fill a quota. The suite is green because the covered logic met its specification on the exercised paths.

Remaining gaps — each named exactly, with the reason and what would be needed. Branch totals include roughly seventy compiler-generated exception and unwind arcs from matrix, filter, and parameter templates; these correspond to no source decision, cannot be driven from the public API, and are reported separately rather than excluded.

Gap	Location	Status	Why it remains	What would be needed
Missing-parameter error log	Battery line 72	Uncovered line (the only one)	Cannot fire with intact v1.17.0 parameter metadata	A corrupted parameter store (scratch-store test), proposed below
Three hysteresis arcs	H-D5/H-D7 sub-cases	Unreachable by construction	The operand combinations contradict the line-75 entry guard (proven by inspection, backed by branch data)	Nothing — no input can reach them
Non-finite-average arm	B-D9 first condition	Unreachable via public API	Its only writer is itself guarded by the B-D10 finiteness check	Nothing through the public API

			— defense in depth, subsumed downstream	
Copy-failure path	Battery line 356	Not deterministically reachable	Requires a topic withdrawal between the update check and the copy inside a single call	Fault-injection at the uORB layer; our unadvertised-topic test documents safe behavior there instead
28 compiler arcs	Gyroscope.cpp call lines (46, 84, 90–129, 196, 248)	Never taken across 34 passing tests	Exception/ unwind arcs on lines whose calls have no source-level branch — same category as Battery's ~70; backed by per-arc branch data	Nothing — no test input drives unwind paths
5 gyro guard arcs	Gyroscope.cpp L60, L88, L161, L231, L243	Unreachable by construction	Same device id always yields the same externality (L60); switch index is loop-bounded to 0–3 (L88); index ≥ 4 unreachable via the public API (L161, L243); a valid desired index equal to the bound index always takes the direct-bind path first (L231)	Nothing — proven by inspection against the setter, Reset, and slot-search code

Limitations of what this evidence can claim. Only four of thousands of PX4 files are covered, so failsafe consumers, estimators, controllers, and drivers are outside the evidence. Estimator convergence was shown only on synthetic steps, never on real noisy flight data. Fixed-wing remaining-time paths used synthetic phase stamps rather than estimator output. All timing is synthetic, so real-time scheduling effects are out of scope. **Improvements proposed:** replay long-horizon current/voltage traces through charge estimation, add the corrupted-parameter scratch-store test, cover the fixed-wing gate with estimator-produced phase data in simulation, and upstream this first-ever Battery functional test so its zero baseline never returns. An independent consistency audit by Taimoor Khalid (audit report, per-arc gap classification, packaging checklist in taimoor/) verified hashes, counts,

patch applicability, and every gap claim against the evidence; each finding was re-verified and either closed with new probe tests or absorbed into this report.

9. Final quality judgment

The structural evidence supports a narrow but strong claim: the four analyzed components behave as specified on every executable path reachable locally. Hysteresis reaches 22/22 lines with all reachable branches taken, including exact timer-expiry boundaries the upstream suite never probes. PID reaches 22/22 lines and 10/10 branch arcs, closing the non-finite-integral and derivative-history gaps upstream leaves open. Battery — which had no upstream test and therefore 0% baseline coverage — reaches 230/231 lines with all 19 functions covered, and independence pairs prove every atomic condition of the failsafe-critical warning, fault, and remaining-time gates can flip its decision, except cases proven unreachable by construction rather than by weak tests. Gyroscope — likewise starting from a 0% baseline with no upstream test — reaches 112/112 lines with all 12 functions covered and every reachable branch taken across 34 tests, discharging its slot-lookup, validity-guard, and save-path obligations round by round. Eight student-test failures during development were each traced to wrong expectations, never to production defects, and no reproducible production defect is claimed; the suites are green because the covered logic met its specification on the exercised paths, and no failure was manufactured. Confidence is therefore high but strictly scoped: statement coverage is complete over the analyzed scope and every reachable decision shows both outcomes, so residual risk concentrates in the individually justified gaps — the L72 corrupted-param log, three hysteresis arcs contradicting the entry guard, the subsumed B-D9 non-finite arm, and the Gyroscope compiler and by-construction arcs. Everything outside the scope remains unsupported: the failsafe consumers of these signals, the estimators feeding them, real-sensor noise, real-time scheduling, and the rest of the PX4 codebase, none of which this evidence addresses. In particular, RLS convergence was shown only on synthetic steps and fixed-wing remaining-time behavior only on synthetic phase stamps. The honest reading is that the selected business logic is structurally sound and deterministically verified at unit and functional levels, while any claim about PX4 as a whole would require system, simulation, and flight evidence far beyond this assignment.

Methodologically, the suite follows a strict setup–run–check discipline with fresh objects per case, fixed synthetic timestamps, and hand-computed expectations, so

every passing test is a reproducible white-box obligation discharge rather than an observational anecdote.

Word count: 369 (checked programmatically before export).

10. Reproduction instructions

```
# 1. Baseline (inside PX4-Autopilot @ d6f12ad, see §2 for clone/setup)
git clone --branch v1.17.0 --recursive https://github.com/PX4/PX4-Autopilot.git
cd PX4-Autopilot && git rev-parse HEAD # expect
d6f12ad1c4f70ad3230afd7d86e971421e02fef4
bash Tools/setup/ubuntu.sh # PX4 toolchain (sudo; use a space-free path)

# 2. Normal test build + run (all commands from the repo root)
cmake . -G Ninja -DCONFIG=px4_sitl_test -B build/px4_sitl_test
cd build/px4_sitl_test
ninja unit-Hysteresis unit-HysteresisStructural unit-PID unit-PIDStructural
functional-BatteryStructural functional-GyroscopeStructural
./unit-Hysteresis && ./unit-HysteresisStructural && ./unit-PID && ./unit-
PIDStructural && ./functional-BatteryStructural && ./functional-GyroscopeStructural

# 3. Coverage build + measurement (separate build dir, instrumented)
cd ../../ && cmake . -G Ninja -DCONFIG=px4_sitl_test -DCMAKE_BUILD_TYPE=Coverage -B
build/px4_sitl_coverage
cd build/px4_sitl_coverage && ninja unit-Hysteresis unit-HysteresisStructural unit-
PID unit-PIDStructural functional-BatteryStructural functional-GyroscopeStructural
./unit-Hysteresis; ./unit-HysteresisStructural; ./unit-PID; ./unit-
PIDStructural; ./functional-BatteryStructural; ./functional-GyroscopeStructural
lcov --capture --branch-coverage --directory . --gcov-tool gcov --ignore-errors
mismatch -o /tmp/cov_all.info
lcov --branch-coverage --extract /tmp/cov_all.info "*/src/lib/hysteresis/*"
"*/src/lib/pid/*" "*/src/lib/battery/*" "*/src/lib/sensor_calibration/*" --ignore-
errors mismatch -o scope_coverage_branch.info
genhtml --branch-coverage scope_coverage_branch.info -o html
```

11. AI-assistance record

AI tooling assisted repository navigation, build troubleshooting, test scaffolding, and coverage analysis throughout, but every AI-suggested test, expectation value, and coverage claim was independently verified by compiling, running, and measuring — eight wrong AI-assisted expectations were caught and corrected during development (two Battery reset-gate timings, one PID NaN output, two Ashar PID sign expectations refuted by PID line 47, and three Ashar saturation expectations refuted by the absence of a freeze gate in lines 57–64). Scope, MC/DC pairs, and gap justifications were derived from direct source reading rather than model assertions. Ashar's record: OpenCode and Muse Spark for navigation, float32 verification scripting, scaffolding, and handoff drafting, with introduced assumptions (brief line and decision framing, fallback sign expectations) corrected through direct source reading, hand recomputation, and an independent float32

model. Taimoor's record: OpenCode for the cross-file consistency audit, coverage-gap classification from raw branch data, report prose drafting, viva-bank derivation, and the packaging checklist, writing no PX4 or test code; his line-to-arc mapping risk was contained by citing raw branch-data strings, and every one of his findings was re-verified before merging. The Gyroscope suite (34 tests, three coverage rounds, per-arc gap classification) was AI-scaffolded under lead direction and verified the same way — every expectation compiled, run green, and measured, with the branch-data file as the authority for each gap claim. No raw chat transcripts are submitted, and no production code was AI-modified — the diff contains only test registrations and new test files. The full per-iteration log is `AI_ASSISTANCE_LOG.md`.

12. References and evidence index

- Brief: `Assignment02-SQE.docx.md` · Plan: `WORKFLOW_PLAN.md` · Progress: `PROGRESS_LOG.md`
- Workbook: `workbook/` (proposed submission copy: `Workbook_proposed.xlsx`, 2 sheets) · Patch: `patch/` · Coverage: `evidence/baseline/`, `evidence/final/` · Logs: `evidence/logs/` · Teammate deliveries: `ashar/`, `taimoor/`
- Official: PX4 Repository, Developer Environment, Unit Tests, Testing & CI, Simulation, v1.17 Release Notes (per brief).